

Two-Tier Reinforcement Learning with an Oracle-Free Top- K Search for Deep Learning Job Scheduling on Heterogeneous GPU Clusters

Ali Ahmadi Esfidi

Department of Mathematics and Computer Science, Amirkabir University of Technology

ali.ahmadi2004@aut.ac.ir

October 10, 2026

Abstract

Efficient job scheduling is crucial for maximizing the utilization of heterogeneous GPU clusters that run machine learning workloads. Traditional heuristic- or optimization-based methods often fail to adapt when job types or hardware availability change. We present a two-tier reinforcement learning scheduler: an inner agent assigns jobs to accelerators and an outer agent decides whether to duplicate a job onto additional hardware. On the Gavel throughput dataset the complete two-tier system reaches a mean cumulative throughput of 17.91, above the inner agent alone (15.15), a total-throughput greedy oracle (16.10), and Sia (14.85), a state-of-the-art goodput-optimized cluster scheduler—while consuming only observable state at decision time.

Analysing where the inner agent loses to the oracle shows that the deficit is one of *selection among near-ties*: re-scoring just the K most probable actions with the true reward already exceeds the full-space oracle (17.21 at $K=5$). That refinement is not deployable, however, because scoring a candidate requires the co-location throughput of a placement that has not been made. The second half of this paper removes that dependency. We show the reward of a placement is determined by an 18-dimensional sufficient statistic readable from the scheduler’s own bookkeeping; slots sharing it are provably reward-equivalent, so a small weight-shared network over it predicts the oracle’s quantity with $R^2 = 0.99972$. The resulting oracle-free search reproduces the oracle-assisted one on the held-out test sets (17.48 against 17.21), beats the greedy oracle by +1.38 (paired $t = 4.70$), and composes with the duplication tier to give the strongest configuration we measure, 19.76 against 17.91 for the two-tier system alone. Bootstrapped online from realised rewards alone—one label per decision, no prior profiling—it recovers that performance after about 1,500 labels.

We report negative results in the same detail. Learned long-horizon value and Q -heads are dominated by their own estimation error and lose to the myopic score; continual online policy updates cost more in exploration than they return within the horizon tested; and a model that never saw two of the five job families is badly wrong on them ($63\times$ the prediction error) yet barely loses end-to-end throughput, because the policy’s candidate set bounds the cost of mispricing. Training uses randomly generated job sets throughout; the 20 held-out sets are used only to report, and configurations are selected on a separate validation stream.

1 Introduction

Efficient job scheduling is an important challenge in modern computing systems. The rapid increase of machine learning workloads has created a strong need for effective use of heterogeneous hardware resources, especially GPUs. Traditional scheduling methods often rely on heuristics or fixed optimization strategies. While these methods are simple, they do not adapt well to dynamic environments where job requirements and resource availability change over time. As a result, they cannot fully take advantage of the performance potential of large computing clusters [1].

Distributed deep learning (DDL) is powering leaps in NLP, computer vision, and networking. That said, the training process for deep neural networks consumes enormous computational horsepower—most organizations must turn to dedicated GPU clusters [2].

Scheduling in deep learning training is inherently a sequential decision problem, where each action influences future resource availability. Locality sensitivity and dynamic fragmentation further complicate the process, limiting the effectiveness of heuristic-based approaches [3].

New generations of GPUs support Multi-Instance GPU (MIG), which partitions a physical GPU into several isolated instances. This enables multiple workloads or multiple users to run workloads simultaneously on one

GPU, maximizing utilization while ensuring performance isolation and guaranteed quality of service (QoS) [4]. In the context of heterogeneous cluster scheduling, MIG effectively transforms one accelerator into multiple “virtual GPUs,” supporting finer-grained allocation and improving overall system efficiency. With the rapid growth of DDL workloads, understanding the behavior of such jobs in real-world GPU clusters has become increasingly important. Traditional heuristic schedulers often fail to capture the highly dynamic characteristics of these workloads. Key challenges include:

- **GPU Heterogeneity:** Modern clusters contain multiple GPU architectures (e.g., V100, A100, H100), each with different performance characteristics. An effective scheduler must account for these differences to maximize cluster utilization.
- **Space Sharing on a Single GPU:** Technologies such as NVIDIA MIG allow a single physical GPU to be partitioned into several isolated “virtual GPUs,” enabling multiple workloads or users to run simultaneously while preserving performance isolation.
- **Distributed Deep Learning Across Multiple GPUs:** Training a single deep-learning job typically requires data or model parallelism across many GPUs. Schedulers must co-locate these GPUs to reduce communication overhead and maintain high throughput.
- **Unknown or Variable Throughput per Workload:** The actual performance of a workload on each GPU type is often unknown in advance and can vary over time, making accurate prediction and adaptive scheduling essential.
- **Sequential and Dynamic Decision Process:** Job arrivals and completions occur continuously, so every scheduling action affects future resource availability. The scheduler must make fast, online decisions in a constantly changing environment.
- **Combinatorial Complexity and NP-Hardness:** The assignment space of jobs to heterogeneous GPUs grows combinatorially with cluster size, and the objective function is non-separable, placing the scheduling problem firmly in the class of NP-hard problems.

Reinforcement learning (RL) offers a promising direction for this problem. Instead of following predefined rules, RL agents can learn adaptive strategies directly from experience. By observing the system state and receiving feedback from past decisions, an RL-based scheduler can improve over time. Prior work has demonstrated that RL is a powerful framework for tackling this challenge, showing it can significantly increase hardware utilization, reduce job completion times, and improve overall throughput [3, 5, 6].

In this paper, we introduce a dual-agent RL method for job scheduling in heterogeneous GPU clusters. The method is built around two agents that work together. The inner agent assigns jobs to specific GPUs based on workload performance profiles. The outer agent decides whether to duplicate jobs in order to improve overall throughput. This hierarchical structure allows the system to combine local scheduling efficiency with global optimization.

We evaluate our method using the *Gavel* throughput dataset, which provides performance measurements for deep learning workloads on heterogeneous GPU architectures. Both agents are trained with Proximal Policy Optimization (PPO) [7], and we compare the learned schedulers against random placement, two greedy policies derived from *Gavel*, and *Sia* [8], a state-of-the-art heterogeneity-aware, goodput-optimized cluster scheduler.

Finally, we design a joint training phase where both agents are trained at the same time. The inner agent begins from its best pre-trained model, and both agents then refine their policies together. This joint training speeds up convergence and leads to higher cumulative throughput. The results demonstrate that coordinated multi-agent learning is an effective strategy for job scheduling in heterogeneous computing environments.

The second contribution of this paper follows from a diagnosis of the first. Section 7.1 shows that the inner agent’s remaining gap to the max-total oracle is not a gap in information but in *tie-breaking*: because throughput depends only on the accelerator type and the co-located job, many slots have identical value, and a single arg max is an arbitrary choice among near-optimal placements. Re-scoring only the top- K actions the policy proposes, and keeping the best, already surpasses the oracle. But that refinement scores candidates with the true reward—the exact co-location throughput of a placement that has not happened—so it is an offline upper bound, not a scheduler. We therefore ask whether the search survives the removal of the oracle, and show that it does, at no measurable cost in quality.

Contributions.

1. **A two-tier RL scheduler** that separates placement from duplication and surpasses both a greedy throughput oracle and *Sia* while acting from observable state alone (Sections 4–7.2).

2. **An exact, observable sufficient statistic for the placement reward** (Section 5.2), which collapses reward-equivalent slots and reduces the learning problem to a small shared function. Its exactness is verified against the environment rather than assumed.
3. **An oracle-free top- K search** (Section 5.5) that reproduces the oracle-assisted refinement on the held-out test sets, composes with the duplication tier, and can be bootstrapped online from realised rewards with no prior cluster profiling (Section 7.6).
4. **A clean train/select/test separation** (Section 6.1): training on randomly generated job sets, configuration selection on a disjoint validation stream, and reporting on the held-out sets alone—together with the negative results this discipline exposes.

The remainder of the paper is organised as follows. Section 4 presents the two-tier scheduler and its two agents; Section 5 develops the oracle-free search, from the sufficient statistic through the learned components to the online variants; Section 6.1 states the train/select/test separation we hold to throughout; and Section 7 reports the results, including the two components that did not work.

2 Related Work

Efficient job scheduling and resource management in heterogeneous GPU clusters has recently received significant attention.

Luo et al. [1] propose *A-SRPT*, an adaptive shortest-remaining-processing-time-first scheduler for distributed deep learning with mixed parallelisms on GPU clusters. By modeling each job as a communication graph and using a random-forest regressor to predict training iterations, A-SRPT combines a heavy-edge GPU-mapping heuristic with a prediction-assisted SRPT framework to minimize inter-server traffic and total job completion time, achieving up to a 92% reduction in flow time over state-of-the-art baselines.

Debner et al. [9] demonstrate that deep reinforcement learning (DRL) can schedule conditional task graphs in heterogeneous settings. Selvan et al. [10] introduce the *PePR* metric, which evaluates deep learning models based on performance per resource unit, showing that smaller models can outperform large-scale ones in resource-constrained environments.

Efficient resource management in clusters is also explored by Chen et al. [11], who employ a two-tiered optimization framework with DRL to handle capacity guarantees and failures. Building on similar ideas, the GOGH method leverages correlation-based neural estimation and RL to achieve adaptive throughput prediction and enhanced performance in heterogeneous GPU clusters.

Venkataswamy et al. [6] show that pre-training RL agents with offline data from heuristic or oracle policies—as in their *Launchpad* method—enhances convergence speed and efficiency in job scheduling for green datacenters, providing a foundation for our approach’s dual-agent design.

Gao et al. [12] present *Aryl*, a cluster scheduler designed to manage training and inference workloads in GPU clusters. Aryl introduces capacity loaning and elastic scaling to improve GPU utilization and reduce job queuing times, employing heuristics to minimize preemptions and optimize job completion times.

Finally, He et al. [13] propose *CuAsmRL*, which utilizes DRL to enhance SASS schedules, achieving up to a 26% performance gain. This work further motivates our two-tier RL approach for scheduling in heterogeneous GPU clusters.

Closest to our evaluation setting, Jayaram Subramanya et al. [8] propose *Sia*, a heterogeneity-aware cluster scheduler that jointly chooses each job’s GPU type and GPU count. Sia builds a goodput matrix over all (job, configuration) pairs, row-normalizes it so that utilities become comparable across jobs, and solves a single integer linear program per scheduling round under cluster-capacity constraints, with a fairness exponent that trades throughput against equal progress. We use Sia as our principal heuristic baseline because it is the strongest published method that reasons about GPU heterogeneity and multi-GPU allocation in the same optimization.

Collectively, these studies motivate our formulation of job scheduling as a reinforcement learning problem, in which we train PPO agents on the Gavel dataset and compare them against greedy oracles and Sia.

3 Problem Formulation

To formally define the job scheduling problem with duplication, let $\mathcal{S}$ denote the set of servers, and let $\mathcal{A}$ be the set of accelerator types (e.g., different GPU models). Let $\mathcal{J}$ represent the set of jobs active in the cluster, and define $\mathcal{C}$ as the set of valid job combinations. Since we allow at most two jobs to share an accelerator, each combination

Table 1: Important Notations.

Symbol	Definition
$\mathcal{S}$	Set of all servers
$\mathcal{A}$	Set of all accelerator types
$\mathcal{J}$	Set of all jobs
$\mathcal{C}$	Set of job combinations
$\mathcal{C}_j$	Set of job combinations containing job j
$ c $	Number of jobs in combination $c \in \mathcal{C}$
$m_j(c)$	Multiplicity of job j in combination c , $m_j(c) \in \{0, 1, 2\}$
$T_{a,j}^c$	Throughput of job j on accelerator of type a when combination c is assigned
$x_{a,s}^c$	Placement of combination c on accelerator of type a in server s

$c \in \mathcal{C}$ contains either one or two jobs. Let $m_j(c) \in \{0, 1, 2\}$ denote the number of times job $j \in \mathcal{J}$ appears in combination c .

The throughput of job j on accelerator type $a \in \mathcal{A}$ under combination c is denoted by $T_{a,j}^c \geq 0$. This value is measured or estimated from profiling and reflects the performance of j when co-executed with the other jobs in c .

We introduce binary decision variables $x_{a,s}^c \in \{0, 1\}$, where $x_{a,s}^c = 1$ indicates that combination c is assigned to an accelerator of type a on server $s \in \mathcal{S}$, and $x_{a,s}^c = 0$ otherwise.

The goal is to maximize the overall throughput while allowing limited job duplication. Each job must be scheduled at least once and at most twice, ensuring that duplication is possible but bounded.

Problem 1 (GPU allocation problem).

$$\max_{x_{a,s}^c} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \sum_{c \in \mathcal{C}} \sum_{j \in \mathcal{J}} m_j(c) T_{a,j}^c x_{a,s}^c \quad (1a)$$

$$s.t. \quad \sum_{c \in \mathcal{C}} x_{a,s}^c \leq 1, \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A}, \quad (1b)$$

$$1 \leq \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \sum_{c \in \mathcal{C}} m_j(c) x_{a,s}^c \leq 2, \quad \forall j \in \mathcal{J}, \quad (1c)$$

$$x_{a,s}^c \in \{0, 1\}, \quad \forall c \in \mathcal{C}, \forall s \in \mathcal{S}, \forall a \in \mathcal{A}. \quad (1d)$$

The formulation can be summarized as follows:

- **Objective (1a):** Maximize the total throughput across all accelerators by selecting the best set of job combinations.
- **Constraint (1b):** Each accelerator can host at most one combination of jobs.
- **Constraint (1c):** Each job must be assigned at least once and at most twice, enabling controlled duplication.
- **Constraint (1d):** Decision variables are binary.

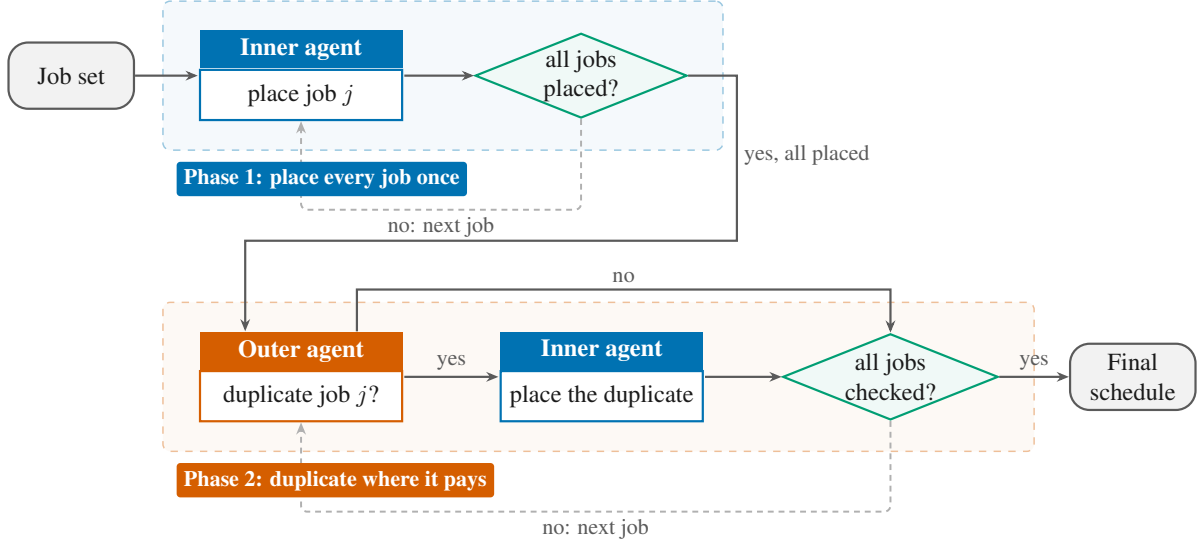
3.1 Computational Complexity of the Assignment Problem

The core assignment problem, even without considering job duplication, is computationally intractable. To understand its complexity, consider a cluster with:

- $\mathcal{J}$: a set of J jobs to assign,
- $\mathcal{S}$: a set of S servers,
- $\mathcal{A}$: a set of A accelerator types *per server* (e.g., K80, P100, V100).

This defines $S \times A$ total GPU “slots.” While assigning one job per GPU is a standard matching problem, the complexity explodes due to two critical factors:

(a) **Co-location of Jobs:** The fundamental unit of assignment is not a single job but a *job combination* $c \in \mathcal{C}$, where a combination can be a single job or a pair of jobs. The size of the set $\mathcal{C}$ grows combinatorially with the number of jobs:



Both blue steps choose a slot by the same rule: greedy arg max, or the oracle-free top- K search.

Figure 1: The two-tier workflow. In phase 1 the inner agent places every queued job exactly once. In phase 2 the outer agent walks the resulting schedule and decides, per job, whether to duplicate it; each duplicate is returned to the inner tier for placement. Both tiers choose a slot by the same rule—the greedy arg max of Section 4, or the oracle-free top- K search of Section 5, which is a drop-in replacement for it.

- Number of single-job combinations: J
- Number of two-job combinations: $\binom{J}{2} = \frac{J(J-1)}{2}$
- Total possible combinations: $|\mathcal{C}| = J + \frac{J(J-1)}{2} = O(J^2)$

The scheduler must choose which of these $O(J^2)$ combinations to place on each of the $S \times A$ GPUs.

(b) Performance Heterogeneity: The throughput $T_{a,j}^c$ of a job j is not a constant value but depends on:

- The *GPU type* a (e.g., a job runs faster on a V100 than on a K80).
- The *job combination* c (i.e., the other job it is co-located with, if any). The performance interference between two jobs is unique to each pair and each GPU type.

3.2 Generalized Scheduling Problem (no duplication, per-job variables)

We eliminate the combination index and introduce binary decision variables

$$x_{a,s}^j = \begin{cases} 1 & \text{if job } j \text{ is assigned to accelerator type } a \text{ on server } s, \\ 0 & \text{otherwise.} \end{cases}$$

$$\max Z = \underbrace{\sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \sum_{j \in \mathcal{J}} T_{a,j} x_{a,s}^j}_{\text{total throughput}} + \underbrace{\sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \sum_{\substack{j < k \\ j, k \in \mathcal{J}}} Q_{a,s}(j, k) x_{a,s}^j x_{a,s}^k}_{\text{pairwise synergy/penalty}} \quad (2)$$

subject to

$$\sum_{j \in \mathcal{J}} x_{a,s}^j \leq 2 \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A} \quad (3)$$

$$\sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} x_{a,s}^j = 1 \quad \forall j \in \mathcal{J} \quad (4)$$

$$x_{a,s}^j \in \{0, 1\} \quad \forall j \in \mathcal{J}, \forall s \in \mathcal{S}, \forall a \in \mathcal{A}. \quad (5)$$

Explanation

- (2): maximize total throughput plus any pairwise interaction bonuses/penalties $Q_{a,s}(j, k)$.
- (3): each accelerator can host at most two jobs.
- (4): each job is assigned *exactly once*.
- (5): decision variables are binary.

This yields a mixed-integer quadratic program (MIQP) equivalent to the earlier formulation but with $|\mathcal{J}| |\mathcal{A}| |\mathcal{S}|$ binary decision variables.

Consequently, the “best” placement for a job cannot be pre-computed in isolation; it depends on its co-location partner and the specific hardware, making the problem inherently non-linear and non-separable. The assignment space grows combinatorially— $O(J^2)$ even before considering hardware types—and the quadratic objective with pairwise interaction terms renders the problem strongly NP-hard. These factors motivate the use of approximate methods, such as reinforcement learning, to obtain near-optimal solutions efficiently.

3.3 Justification for Reinforcement Learning

The problem of job scheduling on heterogeneous GPUs is a combinatorial optimization problem proven to be NP-Hard. While solvers can find near-optimal solutions for small, offline instances, they are unsuitable for large-scale, online cluster environments due to computational intractability, the sequential and uncertain nature of job arrivals, and dynamic system conditions [5, 2].

We employ a Reinforcement Learning (RL) framework because it directly addresses these challenges:

- **Scalability:** Once trained, an RL policy makes fast scheduling decisions via a forward pass through a neural network, bypassing the need for expensive online computation [5].
- **Online Sequential Decision-Making:** RL is inherently designed for making decisions under uncertainty without prior knowledge of the future job queue, optimizing for long-term cumulative reward [14].
- **Robustness to Dynamics:** RL agents learn from experience in a simulated environment, crafting policies that are robust to noise, performance variability [15], and changes in cluster state.
- **Generalization:** A trained agent can generalize its policy to new, unseen job mixes and cluster configurations, unlike solvers which must start from scratch for every new scenario [16].

Therefore, RL provides a powerful framework for learning a fast, adaptive, and high-performance scheduling policy for real-world deployment.

3.4 NP-Hardness of the GPU Allocation Problem

We consider the following formal GPU allocation problem:

$$\max \sum_{s \in S} \sum_{a \in A} \sum_{c \in C} \sum_{j \in J} m_j(c) T_{a,j}^c x_{a,s}^c \quad (6)$$

$$\text{s.t.} \quad \sum_{c \in C} x_{a,s}^c \leq 1 \quad \forall s \in S, a \in A \quad (7)$$

$$1 \leq \sum_{s \in S} \sum_{a \in A} \sum_{c \in C} m_j(c) x_{a,s}^c \leq 2 \quad \forall j \in J \quad (8)$$

$$x_{a,s}^c \in \{0, 1\} \quad \forall c \in C, a \in A, s \in S \quad (9)$$

Step 1: Problem Simplification

Ignoring duplication and interaction effects (i.e., letting $m_j(c) = 1$ and each job assigned exactly once), the problem reduces to assigning jobs to GPUs such that each GPU hosts at most k jobs with additive throughput. This is the well-studied Generalized Assignment Problem (GAP), known to be strongly NP-hard [17].

Step 2: Incorporating Interaction and Duplication

When the interaction term $m_j(c)$ and duplication constraints are included:

- Throughput is non-additive and depends on both job combinations and GPU type.
- Each job can be scheduled once or duplicated, increasing assignment possibilities.

This results in a problem strictly harder than GAP, as the profit from assigning jobs depends on subsets of jobs on each GPU.

Step 3: Proof Sketch

1. Map a known NP-hard problem such as GAP to the above allocation model.
2. In particular, the scheduling subproblem without duplication or interaction exactly reduces to GAP.
3. The full problem allows even more combinations due to non-linear profit functions and duplications.
4. If the GPU allocation problem could be solved in polynomial time, GAP and related problems could as well.

Conclusion: The GPU allocation problem is at least as hard as GAP and, with interaction/duplication, is strongly NP-hard.

4 The Two-Tier Scheduler

This section describes the proposed two-tier reinforcement learning (RL) method for GPU management in heterogeneous clusters. In Subsection 4.1, we present a high-level overview of the scheduler and in Subsection 4.2 we describe components of the scheduler in detail.

4.1 Overview

The scheduler is designed to handle GPU management from two related perspectives. The first perspective is the mapping of ML jobs to GPUs, with the goal of matching each job to the GPU that best fits its requirements and capabilities. The second perspective is the scaling of ML jobs by allocating them additional GPUs. This scaling leverages both the distributable nature of ML workloads and the ability of modern GPUs to run multiple colocated jobs. Rather than optimizing these two objectives in isolation, the scheduler integrates them into a single method. It uses a dual-agent RL approach that coordinates two agents, each responsible for decisions at a different level of scheduling.

Within this method, the scheduler employs two complementary agents. The *inner agent* is responsible for detailed resource allocation, mapping jobs to specific servers and accelerators based on performance considerations. It ensures that each job is placed on the most appropriate hardware. The *outer agent* operates at a higher level, focusing on the scaling of jobs by replicating workers to increase cluster utilization and throughput. This division of responsibilities allows the scheduler to separate fine-grained placement optimization from broader scaling strategies. By combining the two in a coordinated way, the scheduler can learn scheduling behaviors that would be difficult for a single agent to capture effectively.

The overall workflow of the scheduler is shown in Figure 1. When a set of jobs arrives, scheduling begins with the inner agent, which assigns each job exactly once by selecting a suitable server and accelerator. After this initial placement, the outer agent evaluates the resulting schedule with the objective of increasing overall throughput. For each job, it decides whether replication would be beneficial. If replication is selected, the job is returned to the inner agent for assignment to an additional resource. This process continues until the outer agent has reviewed all jobs.

After both the original and replicated jobs are placed, the final schedule is executed on the cluster. The hierarchical structure of the scheduler allows the inner agent to manage fine-grained resource allocation, while the outer agent focuses on high-level optimization to improve scheduling efficiency and throughput.

4.2 Job Scheduling Model

The job scheduling model employs a two-tier RL method with distinct responsibilities for the inner and outer agents.

4.2.1 Inner Agent

The **inner agent** assigns jobs to accelerators, optimizing throughput under co-location constraints.

- **State Space:** Includes:
 - GPU state: a 3D array over servers and accelerators, encoding up to two co-located jobs.
 - Current job features: one-hot model encoding and normalized batch size.
 - Future job statistics: minimum and maximum batch sizes for each model type.
 - Workload progression: the normalized count of remaining jobs.
- **Action Space:** Selection of a (`server`, `accelerator`) pair. Invalid actions are masked to enforce constraints (e.g., at most two jobs per accelerator).
- **Reward Function:** The reward combines the throughput gain from the new job with the change in throughput for co-located jobs:

$$r_{\text{total}} = \frac{\text{Tr}_{\text{new}}}{100} + \frac{\sum_{j \in \text{co-located}} (\text{Tr}_{j,\text{new}} - \text{Tr}_j)}{100}, \quad (10)$$

where normalization ensures stable training.

4.2.2 Outer Agent

The **outer agent** acts after the inner agent has scheduled all jobs once. It decides whether duplicating each job will improve overall throughput.

- **State Space:** Identical to the inner agent’s state.
- **Action Space:** Binary choice for each job:
 - 0: Skip duplication.
 - 1: Duplicate the job and send it to the inner agent for assignment.
- **Reward Function:** Evaluates the throughput impact of duplication using the same formulation as the inner agent.

4.3 Training Procedure

An episode is the scheduling of one complete job set, and follows the workflow of Section 4.1: the inner agent places every job once, then the outer agent walks the schedule deciding which jobs to duplicate, each duplicate being returned to the inner tier for placement. Rewards accrue across the whole episode, so both agents optimise throughput over the entire workload rather than per assignment.

Training proceeds in two phases, which is what allows the contribution of each tier to be attributed separately.

1. **Placement.** The inner agent is trained alone on randomly generated job sets, with no duplication. This yields the placement policy used everywhere in the paper.
2. **Duplication.** The outer agent is then trained on top of it. Initial placements are made by a *frozen* copy of the inner policy, so no change in base placement can be credited to the duplication tier. Duplicates, however, are placed by a second copy of the inner policy that is fine-tuned alongside the outer agent: the outer agent learns *whether* to duplicate while this secondary policy learns *where* a duplicate should go. Training is therefore joint over the duplication decision and the duplicate-placement policy, with the base placement held fixed.

Both phases use PPO with generalized advantage estimation, action masking, and annealed learning rates, clip range and entropy coefficient; the hyperparameters are given in Table 3.

5 An Oracle-Free Top- K Search

The scheduler of Section 4 places each job with a single arg max over the inner policy. This section develops a refinement of that step, shows why its natural form is not deployable, and then makes it so. The duplication tier is untouched throughout: everything here concerns *how a placement is chosen*, so it composes with the outer agent unchanged.

5.1 Top- K Refinement and its Oracle Dependency

The inner policy induces a distribution over the $S \cdot A$ slots. Playing its arg max discards the information in the rest of that distribution. A top- K refinement instead treats the policy as a *proposal*: rank the feasible slots by probability, keep the best K , score each, and play the winner. This pairs the policy’s long-horizon prior—which slots are worth considering at all—with an explicit evaluation of which of them pays now.

Scoring is where the difficulty lies. The obvious score is the environment’s true reward,

$$r(j, s, a) = \underbrace{T(j \mid o, a)}_{\text{own throughput}} + \underbrace{(T(o \mid j, a) - T(o \mid \emptyset, a)) (1 - d(o, s))}_{\text{effect on the co-located job } o}, \quad (11)$$

where T is read from the profiling tables, o is the job already occupying the slot, and d is the distribution discount. Evaluating Eq. (11) requires $T(j \mid o, a)$: the throughput of j given that it will share an accelerator with o . Offline this is a table lookup; online it is unavailable, because the co-location has not happened. A scheduler that scores this way is an upper bound, not a policy—the same objection that applies to the max-total oracle. The following three subsections replace it.

5.2 Slot Canonicalisation

Inspecting Eq. (11) shows that $r(j, s, a)$ depends on the slot only through

$$\kappa(j, s, a) = (a, (\text{type}(o), d(o, s)), d(j, s)), \quad (12)$$

the accelerator type, the co-located job’s type paired with the discount applying to it on server s , and j ’s own discount. Every term is computable from the assignment matrix and the job queue; none requires throughput data. Two slots with the same κ are therefore *exactly* reward-equivalent.

We verify this rather than assume it. Over five random-action episodes, 9,476 feasible slots collapse into 3,737 classes (2.54 slots per class) with a maximum intra-class spread of the true reward of 0.0. On a trajectory the deployed actor plays, 45 feasible slots map to 11.3 classes.

Canonicalisation serves two purposes. It removes redundant work: inside a top-5 candidate list only 2.23 candidates are distinct on average, so 55% of the scorings are skipped with provably identical decisions—the survivor of each class is the member the policy ranked highest, and the members it replaces score identically. And, encoded numerically, κ is exactly the feature vector the reward model consumes.

5.3 Learned Reward Model

Because κ is sufficient, the reward model need not be a $632 \rightarrow 45$ network with one independent output per slot; it is a single shared function of 18 inputs (the current job’s one-hot model encoding and normalised batch size, the accelerator one-hot, the own discount, and the occupant’s presence flag, one-hot, batch size and discount), realised as an MLP with two hidden layers of 256 units—0.07M parameters. This restructuring is what makes the component work: an earlier $632 \rightarrow 45$ model, fitted under the same supervision, agreed with the oracle’s arg max on only 16% of decisions, while the feature-based model reaches 98.3%.

Supervision is offline and dense. Rolling out 800 episodes of a mixture behaviour policy (40% greedy episodes, the remainder ε -exploratory) yields 43,625 visited decisions; at each one the true reward of *every* feasible slot is computed, so a single environment step supplies ~ 45 labels rather than one. The tables act here exactly as logged cluster measurements would in a real deployment: they train the model and are never consulted afterwards. Section 7.6 removes even this offline pass.

5.4 Long-Horizon Heads and the Exact Next State

Two optional components price the future cost of consuming a slot, which the myopic oracle cannot do at all. A value head $V(s)$ is fitted by Monte-Carlo regression on returns-to-go from the greedy episodes only, so that it evaluates the policy that is actually deployed. A Q -head $Q(s, \cdot)$ is trained by double DQN [18] with a Polyak-averaged target; each environment step contributes the played transition plus two off-policy transitions.

Scoring $r + \gamma V(s')$ needs the observation that *would* follow a candidate placement. Rebuilding it costs 0.333 ms, which would dominate the decision. But the observation transition is pure bookkeeping—it never touches the throughput tables—so it can be applied as a delta: one GPU slot gains the job’s encoding, the affected occupancy, server load and model-diversity entries move by fixed amounts, and the queue statistics advance to precomputed suffix values. The result is exact to float32 (maximum absolute deviation 5.96×10^{-8} , checked over 8,011 candidate states) and costs 0.013 ms, a $24\times$ reduction.

5.5 The Oracle-Free Search

A decision proceeds: (i) the frozen inner policy ranks the feasible slots; (ii) the top K are deduplicated by κ ; (iii) the survivors are scored by one of

$$\underbrace{\hat{r}}_{\text{myopic}}, \quad \underbrace{\hat{r} + \gamma V(s')}_{\text{one-step lookahead}}, \quad \underbrace{Q(s, a)}_{\text{action-value}}, \quad \underbrace{(1 - \beta)(\hat{r} + \gamma V(s')) + \beta Q}_{\text{blended}};$$

(iv) the best-scoring candidate is played, with scores within a tolerance of the maximum treated as tied and resolved by the policy’s own ranking. That last detail matters: reward-equivalent placements are not future-equivalent, and the policy’s prior over server identity is a better tie-break than floating-point noise in the last digit of a score.

Because the search only replaces *how a placement is chosen*, it composes with the duplication tier unchanged: the outer agent still decides whether to duplicate, and the search places both the original and, optionally, the duplicate.

5.6 Bootstrapping the Reward Model Online

The offline fit of Section 5.3 assumes a cluster profiled in advance. Dropping that assumption, the reward model can instead be learned on live traffic under deployment rules: one label per decision—the realised reward of the slot actually played, never the ~ 45 counterfactuals—and no table access while deciding. The policy stays frozen throughout, so any change in performance is attributable to the reward model alone. We evaluate two starting points: random weights (*cold start*), and a fit that never saw two of the five model families (*partial*).

5.7 Continual Online Learning

Finally, all four learned components can keep updating during deployment: the reward model supervised on realised rewards, the value head by TD(0), the Q -head by double DQN, and the critic and policy by clipped PPO. Because an unguarded online learner is a reliable way to break a working scheduler, the implementation carries five safeguards: gradients touch only *candidate* networks while acting always uses the *deployed* ones; a candidate is promoted only after passing a gate, either a simulator replay of recent job sets or a simulator-free canary that serves a random slice of live traffic with the candidate and compares arms over the same window; the PPO update stops as soon as the measured $\text{KL}(\text{deployed} \parallel \text{candidate})$, evaluated in eval mode over the whole rollout after every minibatch, exceeds a target; learning rates are reduced and gradients clipped; and action masking is applied at every step, so no amount of learning can emit an infeasible placement.

6 Experimental Setup

We base our experiments on the `Gavel` throughput dataset [19], which provides performance statistics of machine learning workloads executed on heterogeneous accelerators. The raw throughput traces were obtained from the official `Gavel` repository¹ and processed into a structured format suitable for RL experiments. To ensure data quality, we first removed malformed or incomplete entries.

Each workload was standardized into a `(model, batch size)` tuple, ensuring uniform representation across experiments. Throughput measurements were aggregated and stored in a JSON-based schema, which was integrated directly into the RL environment. This preprocessing step ensures that the agents receive consistent state–reward signals while preserving the original job characteristics and hardware constraints.

From this dataset, we constructed a job table encompassing deep learning models across multiple architectures. Each job is parameterized by its model type and batch size, and its execution was profiled on three GPU types—K80, P100, and V100—to capture realistic heterogeneity. The resulting job table is summarized in Table 2.

Training was performed in a simulated environment that generates random job sets of size 20–90, enabling the agents to learn scheduling strategies that generalize across diverse workloads. For evaluation, we used a separate environment consisting of 20 fixed job sets sampled from a predefined dataset directory. These held-out sets are never used during training and serve exclusively for periodic evaluation and final reporting, ensuring that the reported results are not influenced by training data. All experiments were conducted on a MacBook Pro equipped with an M3 chip, providing sufficient computational resources for training and evaluation.

¹<https://github.com/stanford-futuredata/gavel>

Table 2: Deep learning workloads used to construct the job table.

Model	Batch Sizes
ResNet-18	16, 32, 64, 128, 256
ResNet-50	16, 32, 64, 128
Transformer	16, 32, 64, 128, 256
Language Mode	5, 10, 20, 40, 80
Recommendation	512, 1024, 2048, 4096, 8192

All models were trained using the Adam optimizer with a batch size of 128 and a hidden layer size of 1024 across four layers. The inner agent was trained for **15,000 episodes** and the outer agent for **10,000 episodes**. The inner agent was evaluated every 300 episodes and the outer agent every 20 episodes on the evaluation environment to monitor convergence and performance.

Table 3: PPO hyperparameters. Learning rates and the clip range are annealed linearly in log-space over training; the outer agent reuses the inner agent’s settings unless noted.

Hyperparameter	Inner agent	Outer agent
Actor learning rate	$5 \times 10^{-4} \rightarrow 10^{-4}$	$5 \times 10^{-4} \rightarrow 10^{-4}$
Critic learning rate	$10^{-3} \rightarrow 2 \times 10^{-4}$	$10^{-3} \rightarrow 2 \times 10^{-4}$
Discount factor (γ)	0.99	0.99
GAE parameter (λ)	0.95	0.95
Clip range	$0.2 \rightarrow 0.1$	$0.2 \rightarrow 0.1$
Entropy coefficient	$0.05 \rightarrow 0.005$	$0.05 \rightarrow 0.005$
Epochs per update	10	10
Minibatch size	128	128
Rollout length	2048 steps	512 steps
Gradient-norm clip	0.5	0.5

The entropy coefficient decays from 0.05 to 0.005 during training to encourage early exploration and stabilize convergence, and the clipped objective anneals its clip range from 0.2 to 0.1. Advantages are computed with Generalized Advantage Estimation ($\lambda = 0.95$) and normalized per update. Invalid actions are masked out of the policy distribution before sampling, so the agents never propose an assignment that violates the co-location limit.

Baselines

We compare against four reference policies. *Random* places each job uniformly at random among the feasible slots. The two greedy policies are derived from Gavel [19]: *max-throughput* always selects the slot yielding the highest immediate throughput for the current job, while *max-total* greedily maximizes the total cluster throughput of the placement, including co-location interference on the jobs already present. Because *max-total* queries the exact true throughput of every candidate slot at decision time, it is an offline upper bound for single-copy scheduling rather than a deployable scheduler, and we refer to it as the *max-total oracle*.

Our principal heuristic baseline is *Sia* [8]. We reproduce its scheduling formulation rather than a greedy approximation of it: for the whole queue we build the goodput matrix G over all (job, configuration) pairs, row-normalize it as $G_{ij} \leftarrow N_i^{\min} G_{ij} / \min_j G_{ij}$, apply the fairness exponent $p = -0.5$, and solve *Sia*’s binary integer linear program

$$\max_A \sum_i \sum_j A_{ij} (r_i G_{ij})^p + \lambda (1 - \|A_i\|_1) \quad (13)$$

subject to $\|A_i\|_1 \leq 1$ and per-accelerator-type capacity constraints, with $\lambda = 1.1$. Every job is scheduled exactly once and never re-allocated, so the restart factor is $r_i = 1$. A configuration requests one or two GPUs; two-GPU configurations are discounted by the very same-server and cross-server factors the environment charges for distribution, playing the role of *Sia*’s sub-linear scaling efficiency. Since *Sia* assumes exclusive GPUs, we report two variants: *Sia exclusive*, in which each GPU hosts a single job, and *Sia shared*, our extension in which each GPU a configuration requests carries a level—solo or shared—and a shared GPU is priced at the job’s mean co-located throughput over the job types in the queue. The shared variant adds the constraints $\text{share}_X \leq 2y_X$ and

$\text{solo}_X + y_X \leq |S|$ per accelerator type X , where y_X counts the GPUs of that type operated in shared mode. Solving the ILP over the full queue gives Sia strictly more information than the online policies it is compared against.

Because the ILP constrains GPU counts but not server locality, a separate placement pass maps configurations to concrete slots, most-constrained configurations first; a pair that cannot be seated on one server is split across two, which happened 19 times in total across the 20 held-out sets (1,018 jobs) for the strongest variant.

6.1 Evaluation Protocol

Every model in this work follows one convention.

- **Training** uses randomly generated job sets of size 20–90. No component is trained on the held-out sets; for the reward model this is enforced by an assertion in the data-collection loop rather than left to convention.
- **Model selection** – choosing between configurations, such as the search’s K , its scoring mode and its canonicalisation mode – uses a separate *validation* family of randomly generated job sets, drawn with a seed used for no training and no reporting. The test sets are never consulted for this.
- **Testing** uses the 20 held-out sets, walked in deterministic order, one episode each, with **greedy masked arg max**: the action mask comes from a single definition of feasibility shared by every policy in the code-base, and the highest-probability feasible action is played. Sampling, dropout and exploration are off. The top- K policies follow the same rule with one extra step—rank by the policy, then pick among the top K by score.

Every number reported below is on the 20 test sets. The configuration that reports them— $K=5$, myopic scoring, deduplicating canonicalisation—is the one validation selects: on 40 validation sets it scores 16.23 against 15.79 for $K=3$, 15.22 for the one-step lookahead, 15.00 for the expanding candidate set and 13.07 for the Q -head alone, the same ordering the test sets show in Section 7.5.

Verifying that the search is oracle-free. We do not rely on code inspection. During every decision the environment’s throughput table and its interference estimator are replaced with objects that raise on any access, restored only to pay out the reward once the action is taken. All four scoring modes complete full episodes with the table armed; the oracle top- K and max-total controls trip the guard on their first decision, which is what makes the negative result meaningful.

7 Results

7.1 Inner Agent

The inner agent, responsible for mapping machine learning jobs to heterogeneous accelerators, was first trained on its own. Figure 2 tracks its mean cumulative reward on the held-out sets, evaluated every 300 episodes across 15,000 training episodes. The agent improves steadily from 5.86 at the first evaluation and converges to a best held-out mean of 15.15, which is the checkpoint used for every result reported below.

Figure 3 compares the mean throughput of every policy. Sia is evaluated in four configurations, obtained by crossing exclusive against shared GPUs and single-GPU against multi-GPU allocations: exclusive (12.48 ± 3.20), exclusive with distribution (12.99 ± 3.97), shared (14.27 ± 2.11), and shared with distribution (14.85 ± 2.29). Both axes behave as the cluster model predicts. Allowing GPU sharing raises Sia by 1.79 without distribution and 1.86 with it, because doubling the number of schedulable slots lets it serve every queued job rather than leaving the tail unscheduled. Allowing multi-GPU configurations raises it by a further 0.51 (exclusive) and 0.58 (shared): once the ILP has served every job, the capacity constraint in Eq. (13) is what decides whether spare slots are worth spending on a second copy. This is a property of the global formulation, not of distribution per se—a greedy rule that duplicates whenever the immediate gain looks positive spends slots that later jobs need and *loses* throughput.

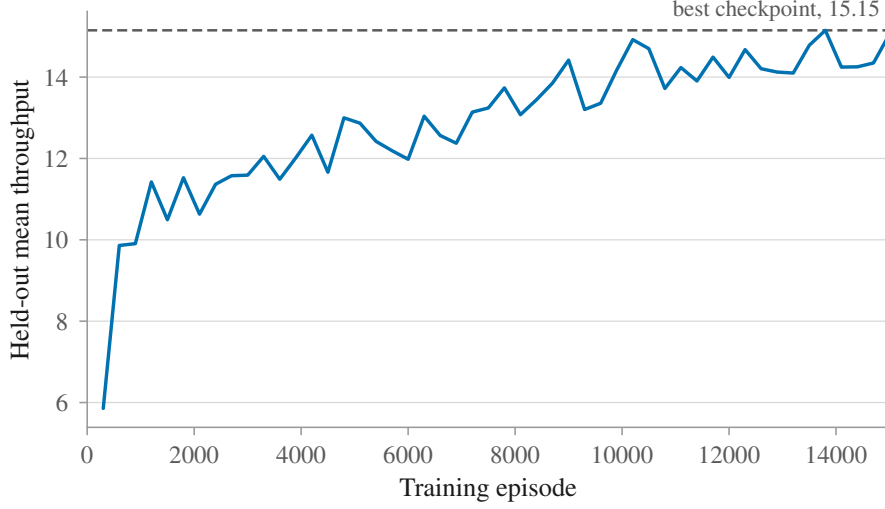


Figure 2: Held-out mean cumulative throughput of the inner PPO agent over 15,000 training episodes, evaluated every 300 episodes. The dashed line marks the best checkpoint, which is the one used for every result reported below.

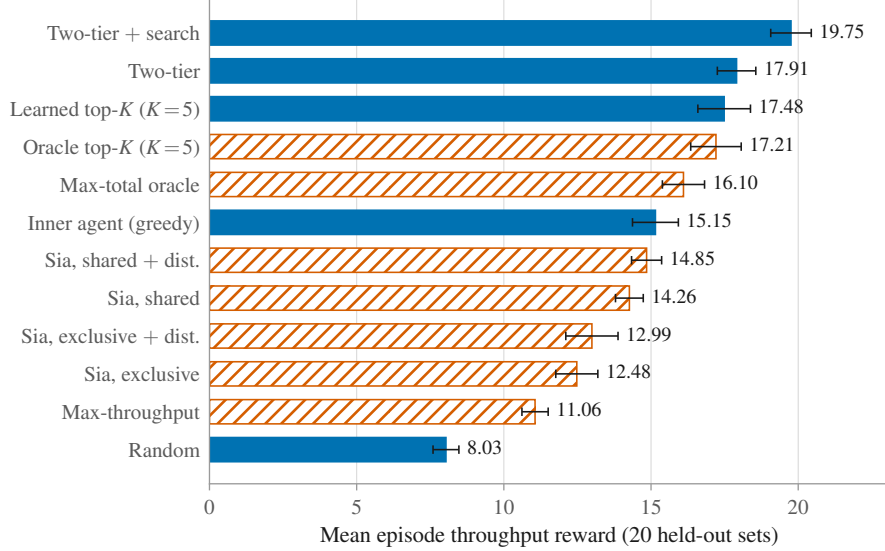


Figure 3: Mean cumulative throughput ($\pm$ standard error) of every evaluated policy over the 20 held-out job sets. Hatched bars read exact throughputs at decision time—the two oracles and the four Sia configurations—and are not deployable schedulers.

The inner agent, used greedily, reaches 15.15 ± 3.48 , above every heuristic baseline—the best Sia variant (14.85 ± 2.29), max-throughput (11.06 ± 1.99) and random (8.03 ± 1.96)—but below the max-total oracle (16.10 ± 3.20). That the learned policy beats Sia while consuming only observable state, whereas Sia solves an ILP over the entire queue with exact throughput profiles, indicates that the gap is not one of search effort: Sia’s normalized, fairness-shaped objective deliberately equalizes progress across jobs, which costs raw throughput on this metric.

The gap between the inner agent and the max-total oracle (15.15 vs. 16.10) reflects *where* each policy obtains its information, rather than a limitation of learning. At decision time the oracle re-scores *every* candidate slot with the exact true throughput of the current job on that accelerator, including pair-specific co-location interference, and takes the maximum. The inner agent, by contrast, sees only the observable system state and takes the single most probable action under its policy. Because throughput depends only on the accelerator *type* and the co-located job—all servers of the same type are interchangeable—many slots have identical true value, so the policy’s distribution over the best slots is nearly uniform and a single argmax amounts to an arbitrary tie-break among near-optimal placements. The resulting per-decision errors are small but compound over the roughly 50 jobs of each job set. The deficit is therefore one of *selection among near-ties*, not of missing information: re-scoring only the top- K actions proposed by PPO and choosing the best among them ($K=3$ and $K=5$) already yields 16.61 and 17.21—*above* the full-space oracle. That refinement is not deployable as stated, since it scores candidates with the true reward; Section 7.3 shows it can be reproduced without one. The oracle resolves near-ties by a fixed enumeration

order that ignores their impact on future jobs, whereas the learned prior breaks ties toward placements that leave the cluster in a better state for the jobs yet to arrive.

7.2 Outer Agent and Joint Training

The outer agent is trained for 10,000 episodes on top of the inner agent, under the two-phase regime of Section 4: the base placement is made by a frozen copy of the inner policy, while a second copy is fine-tuned alongside the outer agent to place the duplicates.

Figure 4 shows both learning signals, evaluated on the held-out sets every 20 episodes. The outer agent’s own episode reward—the throughput its duplication decisions add—rises from -2.13 to roughly $+2.5$, crossing zero early in training: the agent starts by duplicating indiscriminately, which costs throughput, and learns to duplicate only where spare capacity makes it profitable. The total held-out throughput of the resulting two-tier schedule rises in step, from 13.02 to 18.14 at its best evaluation.

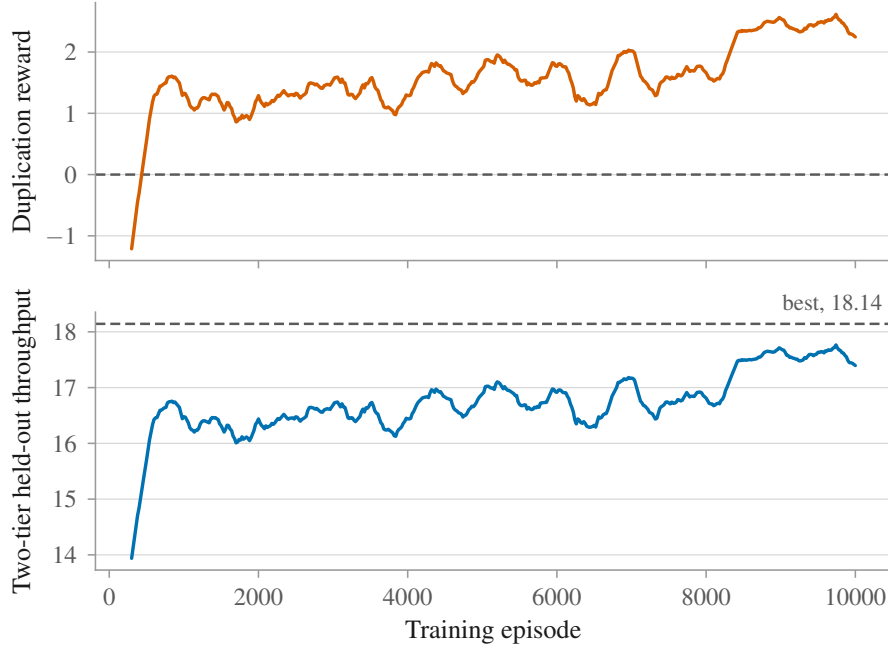


Figure 4: Joint training of the outer agent and the duplicate-placement policy over 10,000 episodes, evaluated every 20 episodes and smoothed with a 15-point moving average. Top: reward from the duplication decisions alone, which starts negative and crosses zero as the agent learns to duplicate only where spare capacity makes it profitable. Bottom: total held-out throughput of the complete two-tier schedule.

Re-evaluating the saved checkpoint end to end gives a mean cumulative throughput of 17.91 ± 2.92 on the held-out sets—an 18% improvement over the inner agent alone (15.15), and above both the max-total oracle (16.10) and the strongest heuristic baseline, Sia with shared GPUs and distribution (14.85 ± 2.29). It beats the oracle on 13 of the 20 held-out sets (Figure 5). These results show that the outer agent’s duplication decisions provide substantial throughput beyond what the placement agent can reach alone, and that freezing the base placement is enough to attribute that gain to the duplication tier.

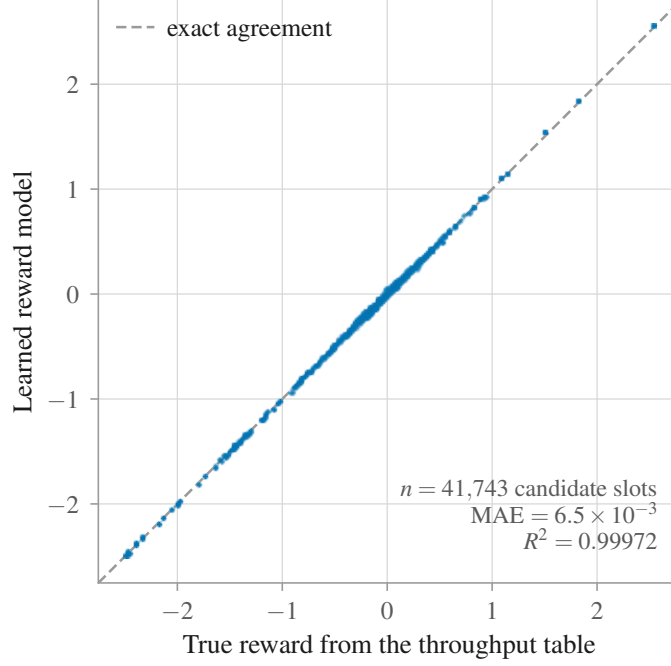


Figure 6: The learned reward model against the oracle quantity it replaces, on every feasible slot of greedy rollouts over the 20 test sets. The throughput tables are read here only to produce the horizontal axis; the model sees none of it, and was fitted on randomly generated job sets.

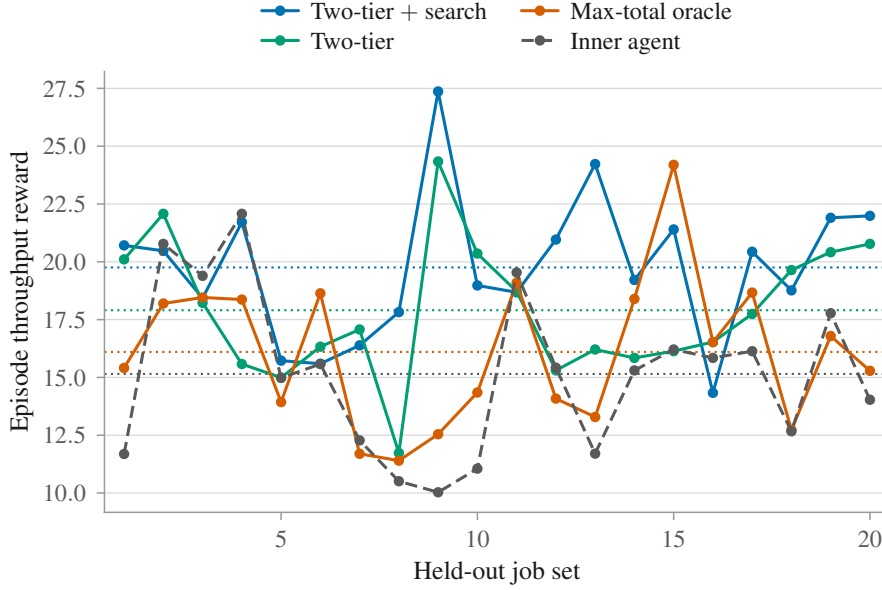


Figure 5: Per-set cumulative throughput across the 20 held-out job sets; dotted lines mark the means. The two-tier system beats the max-total oracle on 13 of the 20 sets, and adding the oracle-free search raises it further on most of them.

7.3 The Oracle-Free Search Matches the Oracle-Assisted One

Figure 6 shows the learned prediction against the true reward for every feasible slot along greedy rollouts on the test sets. Over 41,743 candidate slots the model attains $R^2 = 0.99972$ and a mean absolute error of 6.5×10^{-3} in reward units. On a validation split of its own training distribution it agrees with the oracle’s arg max on 98.3% of decisions, at a mean regret of 4.5×10^{-5} per decision—roughly four orders of magnitude below the scale of the rewards being compared.

Table 4 gives the comparison. The oracle-free search reaches 17.48 ± 4.00 , reproducing the oracle-assisted refinement it replaces (17.21) while reading no throughput data at decision time, and beating the max-total oracle by +1.38 (paired $t = 4.70$) and the inner agent alone by +2.33 ($t = 3.91$). At $K=3$ the two versions are

Table 4: Mean $\pm$ standard deviation over the 20 test sets, with paired statistics against the oracle-free search. Italicised rows read exact throughputs at decision time and are upper bounds, not deployable schedulers.

Policy	Test (20 sets)	Δ	paired t
Two-tier + search (both phases)	19.76 $\pm$ 3.08	+2.27	+1.90
Two-tier + search (primary only)	19.24 $\pm$ 2.93	+1.75	+1.61
Two-tier	17.91 $\pm$ 2.92	+0.42	+0.38
Learned top- K ($K=5$)	17.48 $\pm$ 4.00	—	—
<i>Oracle top-K ($K=5$)</i>	17.21 $\pm$ 3.84	-0.28	-2.06
Learned top- K ($K=3$)	16.65 $\pm$ 3.19	-0.83	—
<i>Oracle top-K ($K=3$)</i>	16.61 $\pm$ 3.18	-0.87	—
<i>Max-total oracle</i>	16.10 $\pm$ 3.20	-1.38	-4.70
Inner agent (greedy)	15.15 $\pm$ 3.48	-2.33	-3.91
<i>Sia, shared + distribution</i>	14.85 $\pm$ 2.29	-2.64	—

likewise level (16.65 against 16.61). *Reproducing* the oracle-assisted result without an oracle is the claim this work supports; the +0.28 by which the learned search edges ahead at $K=5$ is within the noise of a 20-set comparison and we do not read anything into it.

7.4 Composing the Search with the Duplication Tier

The search and the duplication agent are orthogonal: one improves *where each copy goes*, the other decides *how many copies there are*. We therefore leave the outer agent exactly as trained—it still decides whether each job is duplicated—and replace only the placement step, which the two-tier system performs twice per duplicated job.

Which placements the search performs. The scheduler places jobs with two different networks: the frozen primary policy for the initial placement, and the fine-tuned secondary policy for duplicates (Section 4). The search can replace either or both:

	initial placement	duplicate placement	Test (20 sets)
Two-tier	primary, arg max	secondary, arg max	17.91 $\pm$ 2.92
Two-tier + search (primary)	search	secondary, arg max	19.24 $\pm$ 2.93
Two-tier + search (both)	search	search	19.76 $\pm$ 3.08

Replacing both placements gives 19.76 ± 3.08 , which is +1.85 over the two-tier baseline (paired $t = 2.73$, better on 13 of the 20 sets) and +2.27 over the search alone. Using the search for the duplicates as well as the initial placements adds +0.52 over using it for the initial placements only. It also schedules with fewer duplicates (6.1 versus 7.5 per episode): better placement makes duplication less necessary.

The secondary policy becomes redundant. In the configuration above, both phases search over the *primary* policy, so the fine-tuned secondary network is bypassed rather than re-ranked. Searching the duplicates over the secondary policy instead gives 19.78 ± 2.89 , a difference of +0.03 (paired $t = 0.13$). Which policy proposes the candidate slots is therefore immaterial once the search selects among them—the fine-tuning that Section 7.2 credits for part of the two-tier gain buys nothing that the search does not already supply.

The duplicate score is an approximation. The initial phase is exact: no duplicates exist while it runs, so κ fully determines every candidate’s reward. Placing a *duplicate* additionally pays $-d(s)(T_{\text{orig}} + T_{\text{dup}})$, and T_{orig} —the throughput of the copy already running—is not a function of κ . The search therefore ranks duplicate slots by an expression missing one term. It still helps, but this is the one place in this work where the score is knowingly incomplete. In a real deployment T_{orig} is measurable on the running copy and should be supplied; inside the simulator, reading it would mean querying the throughput table.

7.5 Ablations

Figure 7 reports two findings that cut against the design intuition, and we state them as results rather than footnotes.

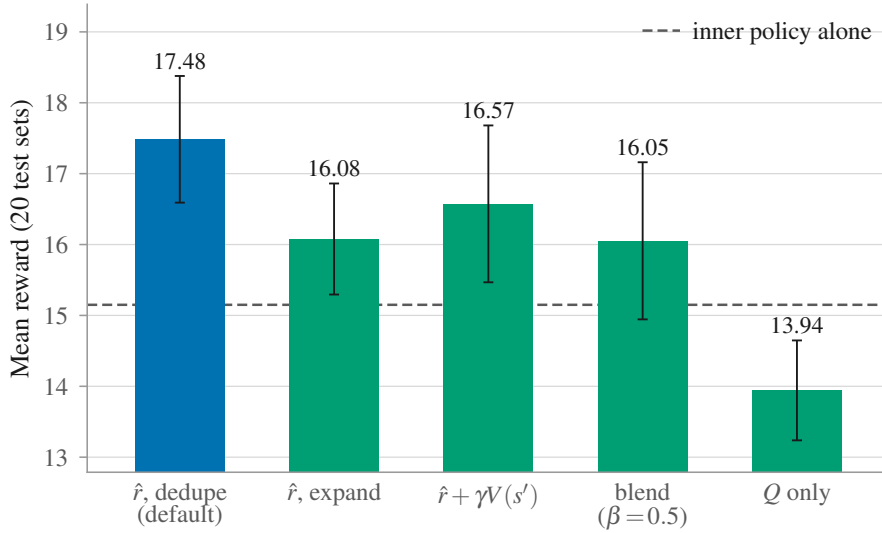


Figure 7: How the candidate set is built and what scores it, on the 20 test sets. Error bars are one standard error. The dashed line is the inner policy used without any search. The winning configuration was selected on validation, not here.

Canonicalisation is a latency tool, not a quality tool. Used to *deduplicate* a fixed top- K list it is free: identical decisions, 55% fewer scorings, confirmed over 150 decisions by an equivalence check. Used to *expand* the list into K distinct classes, it reaches further down the policy’s ranking and scores 16.08 against 17.48. With a myopic score, a wider candidate set simply makes the scheduler greedier; the policy’s probability ordering is itself a long-horizon signal, and widening the set discards it.

The learned long-horizon heads do not pay off at this scale. Every variant that consults them scores below the myopic one: 16.57 for $\hat{r} + \gamma V(s')$, 16.05 for the blend, and 13.94 for Q alone—below even the bare policy. Validation orders them identically, so the choice of the myopic score was made there and merely confirmed here. The reason is a signal-to-noise mismatch. The value head’s validation MAE is ≈ 0.48 and the Q -head’s TD loss plateaus at ≈ 0.10 , while the reward differences between candidate slots are ~ 0.01 – 0.3 . The long-horizon term injects noise one to two orders of magnitude larger than the signal it is meant to refine. The components are implemented, trained and evaluated; the shipped default is myopic.

7.6 Learning the Reward Model Online

A cold start costs about 1,500 labels. From random weights the search scores 12.82—below the bare policy, as expected when the candidate ranking is noise. After 1,461 realised labels, roughly 25 episodes, it reaches 17.16, already past the max-total oracle (16.10); by 7,295 labels it is at 17.38, level with the offline fit (17.48), and it plateaus there (Figure 8). The bootstrap is cheap for the same structural reason the model is small: one shared function of 18 inputs, so every decision teaches it something that transfers to every slot.

The search bounds the damage a bad model can do. The partial model, which never saw ResNet-50 or Transformer, scores 17.33 *before any online update*—almost the full model’s 17.48. This is not generalisation. Measured directly on held-out states, its error on the unseen families is 0.3300 MAE against 0.0052 for the full model, $63\times$ worse, and $8.7\times$ worse than its own error on the families it did see. What rescues the end-to-end score is the candidate set: the top- K are the policy’s own best guesses, which are already good placements, so mispricing among them costs little.

The practical reading is that **the policy sets the quality floor and the reward model sets the ceiling**. The worst case measured—a random model, which picks roughly uniformly among the policy’s top five—is 12.82, and a roughly right model captures most of the available gain. For a component intended to be deployed before it is fully trained, that is the failure mode one wants.

7.7 Online Deployment and Decision Latency

A key advantage of the learned schedulers over the max-total oracle is that they are directly usable as online policies. At decision time, the inner and outer agents consume only the observable system state—per-slot GPU occupancy, the current job’s model type and batch size, and statistics of the remaining queue—and select an

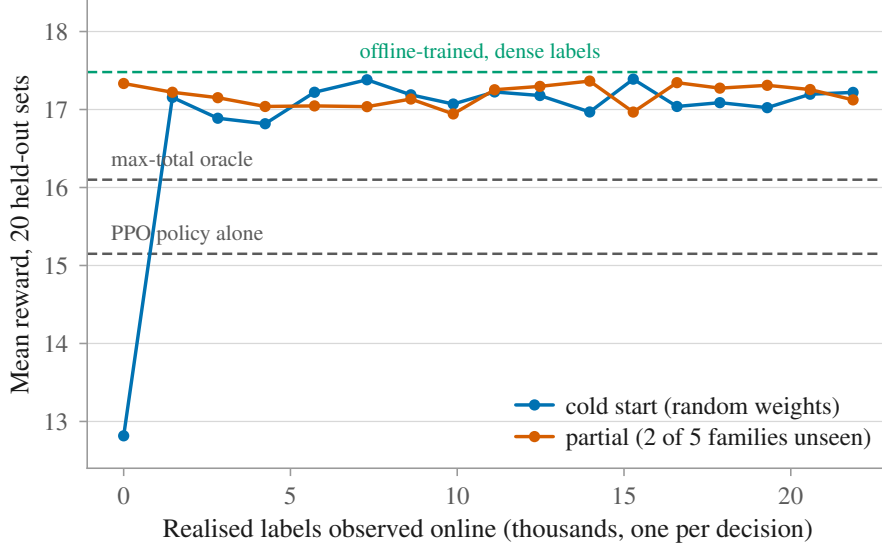


Figure 8: Bootstrapping the reward model on live traffic with one label per decision. The policy is frozen throughout, so the curve isolates the reward model. The oracle top- K reference (17.21) is omitted because it coincides with the curves. Deployment stream: randomly generated job sets. Test: the 20 held-out sets, greedy, never learned from.

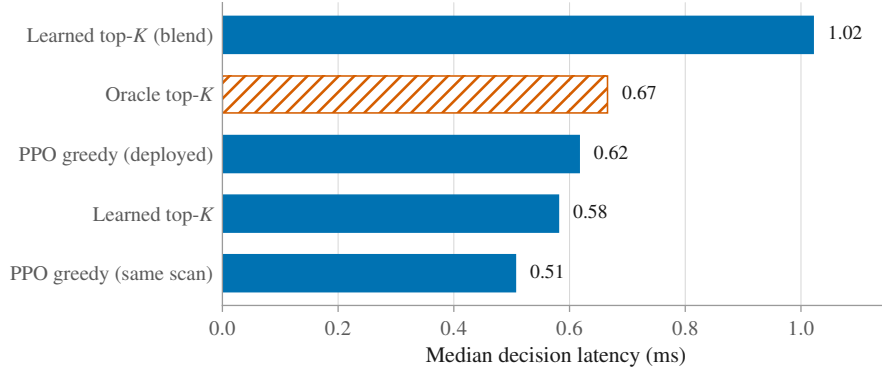


Figure 9: Median latency of a full placement decision, measured on the same half-full cluster state (eager fp32, one CPU thread, 400 timed decisions after warm-up). The hatched bar reads exact throughputs.

action with a single forward pass through their networks. Crucially, no throughput table is queried when choosing an action; throughput estimates appear only in the offline training reward and in the evaluation metric. The measurements below put every decision path on a common cluster state; all of them are far below the seconds-scale inter-arrival times of training jobs in a GPU cluster.

In contrast, the max-total oracle is an offline upper bound rather than a deployable scheduler. Each of its decisions re-evaluates every candidate slot using the exact true throughput of each job on each accelerator type, including pair-specific co-location interference. Such perfect, real-time profiling is precisely the information that is expensive or unavailable in production clusters, which is what motivates learning a scheduling policy from observations in the first place. The inner agent alone therefore lags the oracle slightly (15.15 vs. 16.10), but the complete two-tier system *surpasses* it (17.91 vs. 16.10; Figure 5). Two effects explain this. First, the oracle is *myopic*: it greedily maximizes only the immediate total throughput of the current placement and, when several slots tie at the optimum, resolves the tie by a fixed enumeration order, without considering how the choice shapes the cluster for future jobs. The learned policies are instead trained to maximize cumulative return over an entire job set, so their tie-breaking favours placements that leave resources well-positioned for the jobs yet to arrive. Second, the outer agent’s duplication decisions *enlarge* the action space beyond what any single-placement policy—learned or not—can reach: placing a copy of a job on an underutilized accelerator, or on an accelerator that co-locates favourably with it, converts idle capacity into additional throughput. The oracle never duplicates, so its 16.10 is an upper bound for the *single-copy* scheduling problem; the two-tier system solves a strictly larger problem and exceeds that bound using only observable state and millisecond-scale decisions.

All five placement paths sit within a factor of two of each other (Figure 9). The oracle-free search costs

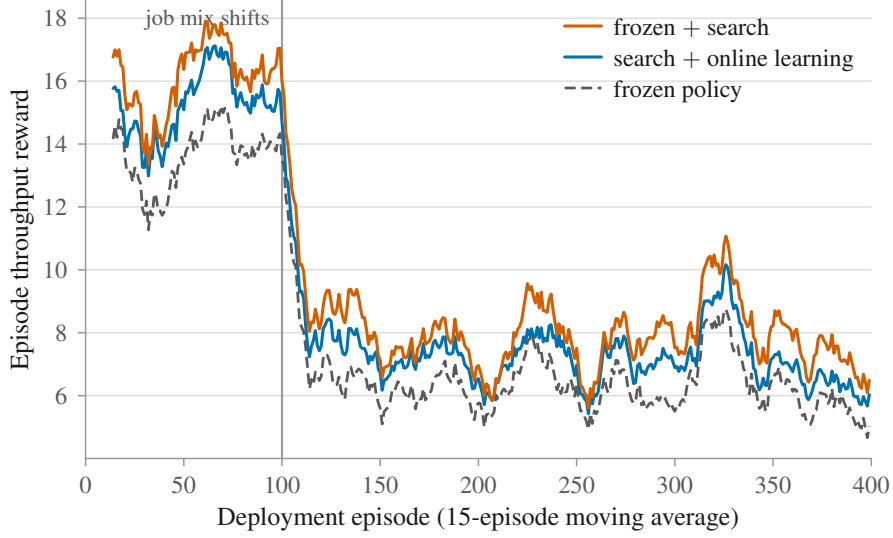


Figure 10: A 400-episode deployment stream whose job mix shifts at episode 100 toward heavier models and larger batch sizes. All arms are served the identical stream, so the comparison is paired.

Table 5: Post-shift results (episodes 100–400). The policy-only probe measures the deployed policy alone—greedy, no search—on a fixed probe set from the current mix, separating what the policy learned from what the search contributes and from the cost of exploring.

	Frozen	Frozen + search	Search + online learning
Realised deployment return	6.25	7.85	7.14
Policy-only probe (mean)	6.45	6.45	6.79 (+5.2%)
Held-out sets (the <i>old</i> mix)	15.15	—	14.19

0.58 ms, against 0.50 ms for a greedy decision built from the same feasibility scan: the search itself adds +0.08 ms, or 15%. Two of its components are negligible—canonicalising all 45 slots costs 0.043 ms and the delta next-state construction 0.013 ms—and the blended rule, which builds K next states and evaluates two additional networks, is the only path past a millisecond. The inner forward pass alone takes 0.11 ms and the outer duplication decision 0.11 ms. All are negligible against the seconds-scale inter-arrival times of training jobs in a GPU cluster.

Latency is not the argument for replacing the oracle, and we want to be explicit about that. The oracle top- K runs in 0.66 ms, only 14% slower than the learned search, because its true-reward evaluation is a handful of dictionary lookups. An earlier version of this measurement reported the oracle at 4.59 ms and the deployed greedy actor at 1.16 ms; both were dominated by an $O(J \cdot S \cdot A)$ feasibility scan that every policy in the codebase shared. Once that scan is vectorised in the one place feasibility is defined, the apparent gap disappears. The case for the learned model rests on availability, not speed: the oracle cannot be evaluated online at any latency, because the co-location it prices has not happened yet.

7.8 Continual Learning under Job-Mix Drift

Figure 10 and Table 5 give the drift experiment. The headline is that **the search, not the learning, absorbs the drift**: with frozen weights throughout, it lifts post-shift throughput from 6.25 to 7.85 (+25.6%). The learned reward model generalises over slot features—job type, accelerator, co-located job—rather than over the job mix, so it stays accurate when the mix changes.

Continual learning on top is a net negative over this horizon (7.14 versus 7.85). The policy it builds does improve: Figure 11 shows the policy-only probe climbing to +14% against the frozen policy before falling back. But roughly 10% of steps spent exploring instead of searching costs more than the policy gains within 300 episodes, and adapting to the new mix costs performance on the old one (15.15 $\rightarrow$ 14.19 on the held-out sets), which is ordinary catastrophic forgetting.

Decomposing the regression on the old mix separates the two learned pieces:

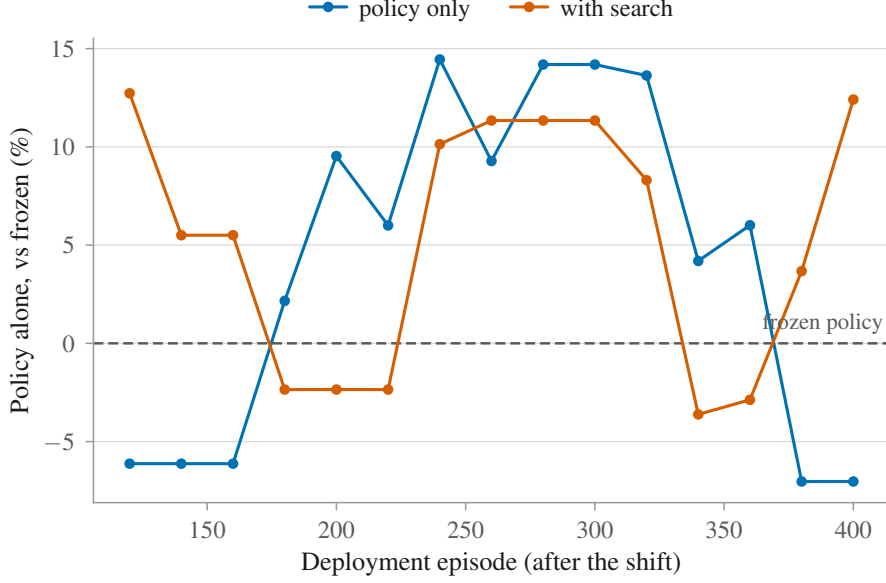


Figure 11: The deployed policy measured alone—greedy, without the search—on a fixed probe set from the drifted mix, relative to the frozen policy on the same sets. The oscillation is the promotion gate: shadow evaluations on eight job sets have limited statistical power, so regressions occasionally pass.

Policy	Reward model	Held-out mean
frozen	frozen	17.48
frozen	online-updated	17.26
online-updated	frozen	15.82
online-updated	online-updated	15.95

The reward model survives online updating—despite seeing one label per decision instead of 45, it costs only -0.22 , and its mean regret on the old mix is in fact lower than the offline model’s (2.7×10^{-4} versus 5.0×10^{-4}). The -1.66 comes from the policy. The practical reading is to update the heads continuously and gate policy updates far more conservatively than the defaults used here. The gate itself behaves as designed—18 of 40 candidates promoted with 2 rollbacks in the policy-only run, 15 of 40 with 4 rollbacks with the search—and its statistical power is the binding constraint on adaptation speed: doubling the monitor set from 8 to 16 job sets raises the probe gain from $+4.1\%$ to $+7.2\%$ and simultaneously deepens forgetting ($14.74 \rightarrow 13.90$).

8 Discussion

Two aspects of these results deserve separate treatment: what our evaluation procedure does and does not establish, and why two of the components we built did not earn their place.

8.1 What the Selection Procedure Costs

Two of the systems compared here have their checkpoints chosen by a score on the test sets: the inner agent keeps the checkpoint with the best held-out evaluation, and the outer agent keeps the best of ~ 500 evaluations taken every 20 episodes across 10,000 episodes. That is selection on the reporting set, and it inflates those two numbers by an amount we cannot measure from the test sets alone.

Scoring the same systems on 40 validation sets—randomly generated, used for no training and no reporting—indicates that the inflation is real and unequal. The two-tier system leads the oracle-free search by $+0.42$ on the test sets but trails it by -0.56 on validation, a reversal consistent with its 500-draw selection procedure; the search’s own margin over the max-total oracle, by contrast, holds at $+1.04$ ($t = 5.60$). We therefore selected every configuration introduced in this paper on validation (Section 6.1) and report the two-tier numbers as they stand, noting that the comparison between the two-tier system and the search is the one that should not be read as settled.

8.2 Why Two Components Did Not Pay Off

The learned long-horizon heads and the continual policy updates were both built on sound reasoning and both lost. It is worth saying why, because the two failures have the same shape.

In each case a learned quantity was asked to resolve a distinction finer than its own error. The value head estimates returns to within ≈ 0.48 and the Q -head’s temporal-difference loss settles near 0.10, while the reward differences separating candidate slots are 0.01–0.3; consulting them therefore injects noise one to two orders of magnitude larger than the signal it is meant to refine (Section 7.5). The continual policy updates fail in the same way at the level of the deployment loop: a promotion gate judging candidates on eight job sets cannot reliably detect the improvements the updates produce, so regressions pass and gains are discarded, and the exploration needed to generate the updates costs more throughput than they return (Section 7.8).

The contrast with the components that did work is instructive. The reward model succeeds because it predicts a quantity that is a deterministic function of an observable, low-dimensional statistic—so its error is small relative to the distinctions it must make—and canonicalisation succeeds because it is exact by construction rather than estimated at all. Where a scheduling decision hinges on a quantity that can be identified exactly, identifying it beats learning it; where it hinges on a long-horizon estimate, the policy’s own ranking, trained on full episode returns, is a more reliable carrier of that information than a separately learned value function.

9 Limitations

“Oracle-free” describes decision time, not training. The offline reward model is supervised by the throughput tables and is, in effect, a learned compression of them. That is the intended reading—a deployment profiles its own cluster offline and then schedules without querying—but it means the contribution is the removal of a runtime dependency, not of the information itself. Section 7.6 removes the offline profiling too: a cold start recovers the same performance from about 1,500 realised labels. It also settles the extrapolation question in the negative—the model does *not* transfer to job families it never saw ($63\times$ worse prediction error)—and shows why that matters less than expected, since the policy’s candidate set bounds the cost of mispricing.

The duplicate-placement score is incomplete. As noted in Section 7.4, T_{orig} is missing from the feature vector. The measured effect is positive, but the component is an approximation where the rest of the method is exact.

Single seed. The learned heads were fitted once, with one seed, and the two PPO agents were each trained once. The evaluation protocol controls for selection over *configurations*, not over training runs; the paired statistics quantify variation across job sets only.

Static queue. Each episode schedules a fixed job set with no arrivals, completions or preemption. Extending the formulation to a continuously arriving queue is the most significant piece of future work.

Simulated cluster. Everything is measured inside the Gavel-derived simulator. Latency figures come from real CPU timings, but the throughput model, the interference model and the distribution discounts are all simulator assumptions.

10 Conclusion

We presented a two-tier reinforcement learning scheduler for heterogeneous GPU clusters, and then removed the one assumption that kept its strongest variant from being deployable.

The two-tier system reaches a mean cumulative throughput of 17.91 ± 2.92 on the held-out sets, above the inner agent alone (15.15), the max-total greedy oracle (16.10), and the strongest Sia configuration (14.85 ± 2.29). Against Sia the learned scheduler wins despite seeing strictly less: Sia solves an integer program over the entire queue using exact throughput profiles, whereas our agents act online from observable state. Against the max-total oracle the gain comes from the duplication tier, which reaches placements no single-copy policy can express.

Diagnosing the inner agent’s remaining deficit as one of tie-breaking led to a top- K refinement that surpasses the oracle—but scores candidates with the true reward, and so is not a scheduler. Making it deployable turned out to need a structural observation rather than a larger model: the placement reward has an exact 18-dimensional sufficient statistic readable from the scheduler’s own bookkeeping, which simultaneously collapses reward-equivalent candidates and reduces the learning problem to a small shared function that predicts the oracle’s

quantity to $R^2 = 0.99972$. The resulting search reproduces the oracle-assisted refinement on the held-out sets, costs 0.58 ms per decision with no throughput lookup—verified by a guard that makes the tables raise on access—and composed with the duplication tier gives the strongest configuration we measure, 19.76. Bootstrapped from random weights on live traffic, it recovers that performance after about 1,500 realised rewards.

Two components did not earn their place, and we report them as such. The learned long-horizon heads are dominated by their own estimation error at this problem scale, and full continual online learning costs more in exploration than it returns within the horizon tested, while degrading performance on the original job mix. The more useful finding is the one that made them unnecessary: because the learned reward model generalises over slot features rather than over the job mix, the frozen search already absorbs a distribution shift severe enough to halve throughput.

Finally, the evaluation protocol proved as consequential as the method. Two of the systems compared here have their checkpoints chosen by a score on the reporting set, and a validation stream that nothing was selected on shows the effect is large enough to reverse one of the comparisons. Every configuration introduced in this paper is therefore selected on validation and reported on the held-out sets, and we recommend the same separation for any future number from this line of work.

Future work could extend the formulation to online scheduling with dynamic job arrivals, explore transfer across cluster configurations, and investigate job preemption and elastic scaling.

Reproducibility

All results are produced by a single evaluation entry point that scores every policy on the same 20 held-out job sets and reports the difference from the committed values, so any drift is visible immediately. The three exactness claims in Section 5—that canonicalisation is reward-preserving, that the delta-built next observation matches the environment, and that the search never reads the throughput tables while deciding—are implemented as executable checks rather than assertions in prose, and each is run as part of the standard verification pass. Every figure is regenerated from the stored measurement data by one script, so no figure can diverge from the numbers it depicts.

Acknowledgments

References

- [1] Ziyue Luo, Jia Liu, Myungjin Lee, and Ness B Shroff. Prediction-assisted online distributed deep learning workload scheduling in gpu clusters. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*, pages 1–10. IEEE, 2025.
- [2] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. MLaaS in the wild: Workload analysis and scheduling in Large-Scale heterogeneous GPU clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960, 2022.
- [3] Yunteng Luan, Xukun Chen, Hanyu Zhao, Zhi Yang, and Yafei Dai. Sched²: Scheduling deep learning training via deep reinforcement learning. In *2019 IEEE Global Communications Conference (GLOBECOM)*, pages 1–7. IEEE, 2019.
- [4] NVIDIA. Dividing nvidia a30 gpus and conquering multiple workloads, 2021. Accessed: 2025-09-12.
- [5] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication*, pages 270–288, 2019.
- [6] Vanamala Venkataswamy, Jake Grigsby, Andrew Grimshaw, and Yanjun Qi. Launchpad: Learning to schedule using offline and online rl methods. *arXiv preprint arXiv:2212.00639*, dec 2022. Version 2: December 2, 2022.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [8] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, and Gregory R. Ganger. Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP ’23)*, pages 642–657, 2023.

- [9] Anton Debner, Maximilian Krahn, and Vesa Hirvisalo. Scheduling conditional task graphs with deep reinforcement learning. In Tetiana Lutchyn, Adín Ramírez Rivera, and Benjamin Ricaud, editors, *Proceedings of the 5th Northern Lights Deep Learning Conference (NLDL)*, volume 233 of *Proceedings of Machine Learning Research*, pages 46–52. PMLR, 09–11 Jan 2024.
- [10] Raghavendra Selvan, Bob Pepin, Christian Igel, Gabrielle Samuel, and Erik B Dam. Pepr: Performance per resource unit as a metric to promote small-scale deep learning in medical image analysis. *arXiv preprint arXiv:2403.12562*, 2024.
- [11] Chang-Lin Chen, Hanhan Zhou, Jiayu Chen, Mohammad Pedramfar, Vaneet Aggarwal, Tian Lan, Zheqing Zhu, Chi Zhou, Tim Gasser, Pol Mauri Ruiz, et al. Two-tiered online optimization of region-wide datacenter resource allocation via deep reinforcement learning. *arXiv e-prints*, pages arXiv–2306, 2023.
- [12] Wei Gao, Qinghao Hu, Zhisheng Ye, Peng Sun, Xiaolin Wang, Yingwei Luo, Tianwei Zhang, and Yonggang Wen. Deep learning workload scheduling in gpu datacenters: Taxonomy, challenges and vision. *arXiv preprint arXiv:2205.11913*, 2022.
- [13] Guoliang He and Eiko Yoneki. Cuasmrl: Optimizing gpu sass schedules via deep reinforcement learning. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, pages 493–506, 2025.
- [14] Richard S. Sutton and Andrew G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [15] Jingkan Wang, Yang Liu, and Bo Li. Reinforcement learning with perturbed rewards. *arXiv preprint arXiv:1810.01032*, 2018.
- [16] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM SIGCOMM Conference*, 2019.
- [17] David B. Shmoys and Éva Tardos. An approximation algorithm for the generalized assignment problem. *Mathematical Programming*, 62(1-3):461–474, 1993.
- [18] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.
- [19] Alexey Tumanov, Haichen Wu, Paulo Costa, and Phillip B. Gibbons. Gavel: Towards efficient cluster scheduling with performance isolation. https://github.com/stanford-futuredata/gavel/blob/master/scheduler/physical_cluster_throughputs.json, 2019. [Online; accessed 19-August-2025].